

## Week 2 – Dominic Tanzillo, dpt7

Repo:

<https://github.com/DominicTanzillo/Ops-AI-Student/>

CI: <https://github.com/DominicTanzillo/Ops-AI-Student/actions/runs/26227674826>

CD: <https://github.com/DominicTanzillo/Ops-AI-Student/actions/runs/26227674919>

Write-Up

This deployment takes the pre-provided LightGBM taxi model and provides it to the public via an API hosted on Google's Kubernetes. With this deployment there is a build pathway automated through GitHub Actions. On every push to main, CI runs a PyTest and Docker build. If CI passes, CD is authenticated to Google Cloud using a service account key (stored as a Github secret) and then builds the image and pushes it. It registers this artifact with a tag for "latest" and then has Kubernetes set the image to the new version. Kubernetes maintains two nodes using LoadBalancer such that if one is down, the other is up. For this deploy the API is live at 34.173.128.241 serves four endpoints, namely: `health`, `heatmap`, `forecast`, and `recommendations`.

In this architecture the most important decision comes from how Kubernetes is deployed. Please see the system architecture as a separate document for a detailed illustration of this deployment. The two replicas are done to match the two-node cluster specs, which allows for redundancy without having them both use compute. With the variables maxSurge and maxUnavailable both set to 1 the deployment always has one ready pod even if the other is being used or there is a transition between versions. Memory is capped at roughly six times the average pandas frame provided to ensure I can probe while still serving an API to the public face. These probes are broken down into readiness checks and liveness checks. Every ten seconds with a 30 second initial delay, a readiness check is sent while liveness checks happen every 30 seconds which has a 60 second delay so that slowness here will not restart the whole deployment.

For the Kubernetes and GitHub Actions architecture this is based on course specs. To defend it, the current setup gives up a managed control plane with two nodes and a balancer (via Kubernetes) for a prediction server that is always live at a free cost. A Cloud Run would be simpler for deployment but run up substantial costs. Artifact registry helps to audit the work and is helpful with both Kubernetes and GitHub producing artifacts but would be less needed on cloud deployment.

In the deployment of the code, there were four bugs in the course repo that I might want to mention here. As stated in the repo, much of the debugging here was done assisted with Claude's AI and in length the AI attestation demonstrates how it was used. The requirements.txt was missing scikit-learn which I added. Then RandomForestRegressor was initially indicated for our model which after consulting our TA we updated to the LGBM model. The new requirements,

however, needed to be updated to 4.5.0 for the LGBM model to parse the provided inputs. Then the forecast endpoint as provided had many loops that took 73 seconds on initial deploy. I was able to correct this down to 3.6 seconds which still isn't at the slide requested 100ms but I'm not sure if this can be done without precomputing every request. Finally, the Git LFS pointer cannot be inherited from repo forks, so I needed to add the file as a regular git blob.

The deployed system has a few constraints to address, most importantly the 3.6 seconds for a forecast which is more than 100ms (requested on the slides). The latency comes from calling `'generate_synthetic_demand_current_demand'` 672 times which can be reduced to 168 because that is how many hours there are in a week (rather than across 4 weeks) but this reduction would take it to 900 - 1200 ms which is still far larger than 100ms. With the course's current constraints, I don't see a way of reaching the desired goal without precomputing every value.

To take this work to a production ready version, I would add more smoke tests to our `'cd.yml'` file to have each endpoint assert non-empty responses which could have caught the bugs before my initial two failed deploys. Additionally, duplicating the GitHub-actions across a second account would honor least privilege style of architecture so that multiple accounts add redundancy. Finally, I would consider more security for how keys are stored. Although I used the repo recommended method of GitHub secrets, this could be improved by using the class slide recommended method of Workload Identity Federation. These three actions would allow the system to be better secured for public deployment with many more uses.

My biggest lesson from this assignment is that pod-Ready is a necessary but not sufficient for "the application is working." I observed three different cases where pods passed all probes while the application was broken in ways that only application-level curl would have detected.

Photos / Proofs

```
C:\Users\virtu>curl http://34.173.128.241/health
{"status":"ok"}
C:\Users\virtu> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
demand-api-5895cbfc59-2tl8w         1/1     Running   0           6m54s
demand-api-5895cbfc59-52mh9         1/1     Running   0           6m54s
C:\Users\virtu> kubectl get svc demand-api
NAME      TYPE           CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
demand-api LoadBalancer  34.118.234.127 34.173.128.241 80:32574/TCP     77m
C:\Users\virtu> curl "http://34.173.128.241/api/forecast?zone_id=68&hour=17&dow=4&steps=16&date=2026-01-15"
[{"time_bucket":"2026-05-21T13:27:48.161951","predicted_trips":6.69,"hour":13,"minute":27,{"time_bucket":"2026-05-21T13:42:48.161951","predicted_trips":38.52,"hour":13,"minute":42,{"time_bucket":"2026-05-21T13:57:48.161951","predicted_trips":24.48,"hour":13,"minute":57,{"time_bucket":"2026-05-21T14:12:48.161951","predicted_trips":35.11,"hour":14,"minute":12,{"time_bucket":"2026-05-21T14:27:48.161951","predicted_trips":28.44,"hour":14,"minute":27,{"time_bucket":"2026-05-21T14:42:48.161951","predicted_trips":32.48,"hour":14,"minute":42,{"time_bucket":"2026-05-21T14:57:48.161951","predicted_trips":31.11,"hour":14,"minute":57,{"time_bucket":"2026-05-21T15:12:48.161951","predicted_trips":31.29,"hour":15,"minute":12,{"time_bucket":"2026-05-21T15:27:48.161951","predicted_trips":31.15,"hour":15,"minute":27,{"time_bucket":"2026-05-21T15:42:48.161951","predicted_trips":31.64,"hour":15,"minute":42,{"time_bucket":"2026-05-21T15:57:48.161951","predicted_trips":31.5,"hour":15,"minute":57,{"time_bucket":"2026-05-21T16:12:48.161951","predicted_trips":31.22,"hour":16,"minute":12,{"time_bucket":"2026-05-21T16:27:48.161951","predicted_trips":31.36,"hour":16,"minute":27,{"time_bucket":"2026-05-21T16:42:48.161951","predicted_trips":31.5,"hour":16,"minute":42,{"time_bucket":"2026-05-21T16:57:48.161951","predicted_trips":31.76,"hour":16,"minute":57,{"time_bucket":"2026-05-21T17:12:48.161951","predicted_trips":31.66,"hour":17,"minute":12}]}
```

Photo 1: Health, Pods, and Recommendations

```
C:\Users\virtu> curl "http://34.173.128.241/api/heatmap?hour=17&dow=4&date=2026-01-15&holiday=regular"
{"demand":{"4":0.68,"13":14.48,"24":4.39,"41":4.05,"43":37.59,"45":2.25,"48":31.8,"50":8.76,"68":46.66,"70":6.18,"74":3.75,"75":10.16,"79":22.7,"87":9.98,"88":5.56,"90":26.4,"93":0.38,"100":24.99,"107":30.79,"113":22.3,"114":15.85,"125":6.34,"132":79.24,"137":19.68,"138":53.96,"140":39.06,"141":41.6,"142":52.6,"143":18.16,"144":15.81,"148":7.93,"151":14.66,"158":13.61,"161":102.8,"162":73.49,"163":58.01,"164":38.47,"166":8.53,"170":52.42,"186":58.02,"195":0.07,"209":2.73,"211":14.71,"229":29.97,"230":53.61,"231":22.17,"233":22.31,"234":49.26,"236":83.25,"237":93.66,"238":33.95,"239":50.91,"246":35.66,"249":23.56,"261":9.96,"262":22.68,"263":32.04},"max":102.8,"hour":17,"dow":4,"holiday":"regular","date":"2026-01-15","is_forecast":false,"data_type":"actual","reference_date":"2026-02-14"}
```

Photo 2: Heatmap

```
C:\Users\virtu>curl "http://34.173.128.241/api/recommendations?zone_id=68&hour=17&dow=4&date=2026-01-15&n=3&holiday=regular"
[{"rank":1,"zone_id":70,"name":"East Elmhurst","borough":"Queens","trips":6.2,"demand_score":6,"drive_minutes":3,"est_yield_min":40,"est_yield_max":160,"efficiency_score":6.983363844393592},{"rank":2,"zone_id":88,"name":"Financial District South","borough":"Manhattan","trips":5.6,"demand_score":5,"drive_minutes":3,"est_yield_min":24,"est_yield_max":99,"efficiency_score":4.30448160535117},{"rank":3,"zone_id":87,"name":"Financial District North","borough":"Manhattan","trips":10.0,"demand_score":10,"drive_minutes":3,"est_yield_min":22,"est_yield_max":90,"efficiency_score":3.92237196348445}]
```

Photo 3: Recommendations

```
C:\Users\virtu>kubectl logs deployment/demand-api --tail=15
Found 2 pods, using pod/demand-api-5479c59f8f-vw8ck
Defaulted container "demand-api" out of: demand-api, download-data (init)
INFO:      10.4.1.1:32906 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:55172 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:54450 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:46324 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:46330 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:55318 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:57438 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:40016 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:40032 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:43058 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:53964 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:55542 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:55552 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:47484 - "GET /health HTTP/1.1" 200 OK
INFO:      10.4.1.1:38474 - "GET /health HTTP/1.1" 200 OK
```

Photo 4: CI and CD confirmation of running

```
C:\Users\virtu>gcloud container clusters delete operationalizing-ai --zone us-central1-a --project ops-ai-dpt7
The following clusters will be deleted.
- [operationalizing-ai] in [us-central1-a]

Do you want to continue (Y/n)?  Y

Deleting cluster operationalizing-ai...done.
Deleted [https://container.googleapis.com/v1/projects/ops-ai-dpt7/zones/us-central1-a/clusters/operationalizing-ai].
```

Photo 5: Deletion and confirmation of cluster